

Introduction to Computer Systems (ICS)

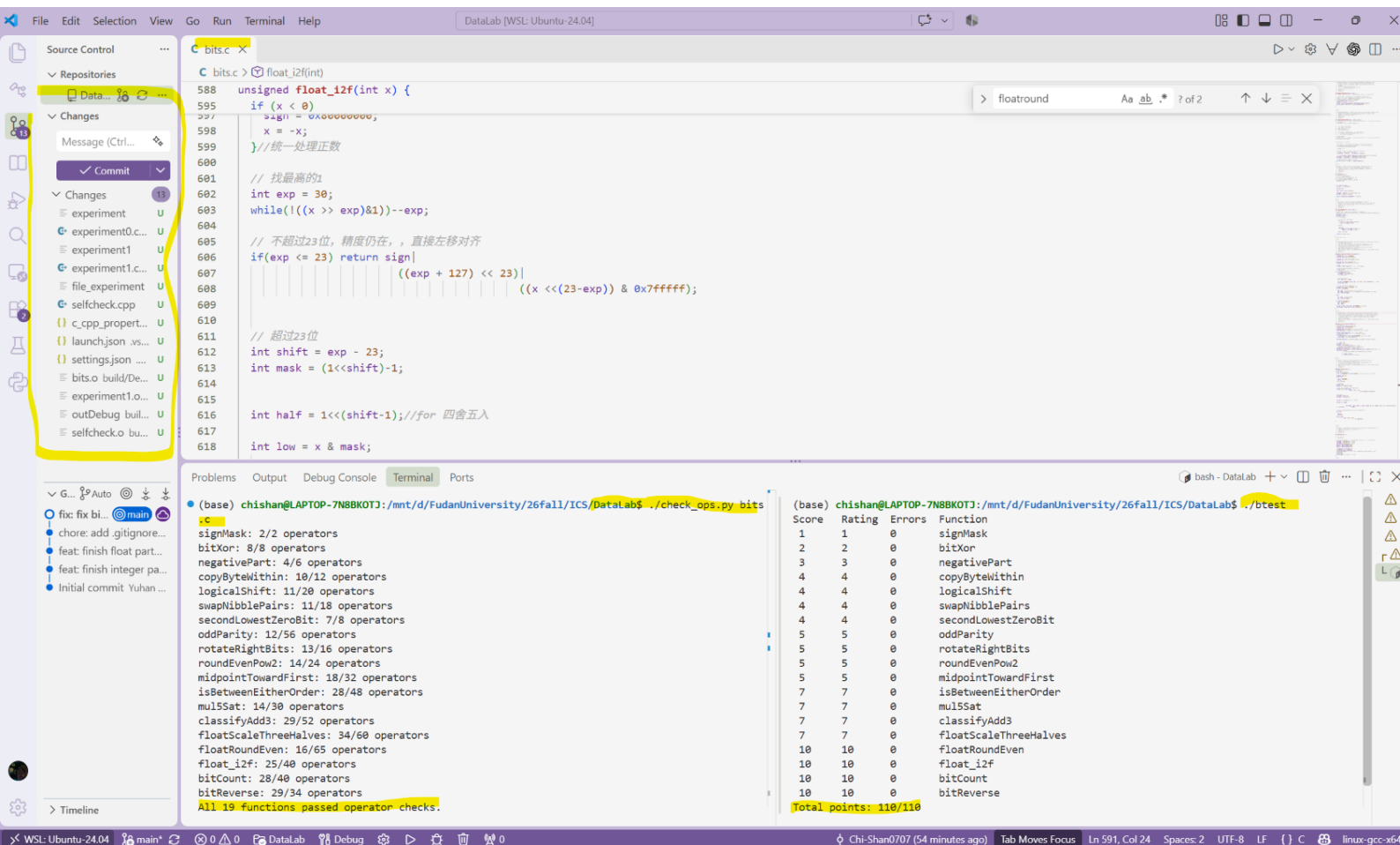
DataLab Report

Fudan University · Fall 2026

Homepage: <https://chi-shan0707.github.io/>

Repo: [Chi-Shan0707/DataLab](https://github.com/chi-shan0707/DataLab)

此为截图，包含 git 记录和两个校验脚本的正确运行。



P1 signMask

```
// P1
// ac
/*
```

```

    * signMask - return a mask with only the most significant bit set (0x80000000)
    *   Legal ops: ! ~ & ^ | + << >>
    *   Max ops: 2
    *   Rating: 1
    */
int signMask(void) {
    return (~0)<<31;
}

```

思路：既然已经给出了目标 (0x80000000) ，

我就可以采用任意一种方法构造。 这里是为了故意凑到 2 个 op 然后利用补码获得（充分多的）1，再左移到最高位（顺便舍弃掉不用的 1）。

P2 bitXor

```

// P2
// ac
/*
    * bitXor - x^y using only ~ and &
    *   Example: bitXor(4, 5) = 1, bitXor(7, 7) = 0
    *   Legal ops: ~ &
    *   Max ops: 8
    *   Rating: 2
    */
int bitXor(int x, int y) {
    // return (x&!y)|(!x&y);
    // return (x&y) &((~x)&(~y));
    return ~(~(x&(~y)) & ~(y&(~x)));
}

```

思路：如同我的注释所写，我一开始想用这个非，但是没有非。

首先，我们 xor 是一种不进位加法。 ~和&都可以 bitbybit，所以我们可以假定 x, y 从 0, 1 取值（即只考虑一位）

Xor 一个很重要的性质就是对称性，我只用研究一边，并且肯定中间得是用 & 来连接，说明如果是 00，11，则两边至少有一边是 0；01 或 10，两边则都得是 1

不难想到，如果是 1 0 / 0 1 一个取反后再与另一个 & 可能得到 1，而 00 和 11 则一定得到 0

如果有一个或就好了！然后我们对于这样的结果取反，再与，则是一个或的同构运算。

进而，比较运算结果后，可以得到 $\sim(x \& (\sim y)) \& \sim(y \& (\sim x))$ 外还需要再来一个取反

P3 negativePart

```

// P3
// ac
/*
    * negativePart - return -x if x < 0, otherwise return 0

```

```

*   Examples: negativePart(-10) = 10, negativePart(5) = 0
*   Legal ops: ! ~ & ^ | + << >>
*   Max ops: 6
*   Rating: 3
*/
int negativePart(int x){
    return  (~(x>>31)&x))+1;
}

```

思路： Easy 题目。如果是正为 0，如果是负才有值。 不难摸索到， 刚好 $x \gg 31$ 在 x 为非负的时候为 0，负数的时候为 0xffffffff， 全是 1， 可以通过一个与运算迅速区分。 最后要回到正数即是，取 “-”， 即 补码。

P4 copyByteWithin

```

// P4
// ac
/*
 * copyByteWithin - copy byte src of x to byte dst, leaving all other bytes unchanged
 *   Bytes are numbered from 0 (least significant) to 3 (most significant).
 *   You can assume 0 <= src <= 3 and 0 <= dst <= 3.
 *   Example: copyByteWithin(0x11223344, 0, 2) = 0x11443344
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 12
 *   Rating: 4
 */
int copyByteWithin(int x, int src, int dst) {
    // 首先注意这里是 要乘上 8?

    // <<3

    // 我希望能点对点地算增量?
    return (((x>>(src<<3))&0xff)<<(dst<<3))|(x&(~(0xff<<(dst<<3)))));
}

```

思路： 只需要对对应位置进行操作就好了。

$(x \gg (src \ll 3)) \& 0xff$ 是一个典型的利用位移和小数字（小于 255）的与来取出对应 source 数码的方法。最后需要再移位加入。

注意这里由于一开始是 $\&0xff$ ，所以整个东西是很干净的。 我只需要 | 上一个在 dst 位置清理过，即 $x \& (\sim(0xff \ll (dst \ll 3)))$ 即可

P5 logicalShift

```

// P5
// ac
/*
 * logicalShift - shift x to the right by n bits, using a logical shift
 *   Can assume that 0 <= n <= 31

```

```

*   Examples: logicalShift(0x87654321,4) = 0x08765432
*   Legal ops: ! ~ & ^ | + << >>
*   Max ops: 20
*   Rating: 4
*/
int logicalShift(int x, int n) {
    // 右移，高位 1 我要处理。
    // 我只用确保前面的 0 都会被我消掉。所以只用 前面都 1 后面都是 0 的一串掩码
    //都取反？
    return ((x>>n) ) & (~(((1<<n)+ (~1+1))<<(32-(~n+1))));
}

```

思路：先做算术右移，只有高位的 1 我才要清理，然后，掩码很简单，先构造全 1111，再移位，取反，就能把足够多的 0 送到高位，清除掉不需要的东西。我们的掩码也正如所说，是一个前面都要 1，后面都是 0 的，所以一个好的 idea 就是 $2^k - 1$ 的形式取反

P6 swapNibblePairs

```

// P6
/*
 * swapNibblePairs - swap the low and high 4 bits within each byte of x
 *   Examples: swapNibblePairs(0xAB) = 0xBA
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 18
 *   Rating: 4
 */
int swapNibblePairs(int x) {
    // return
    ((x>>2)&(15+((15+((15+(15+((15+((15+(15<<4))<<4))<<4))<<4))<<4))<<4)) ^ (x&((15+((15+((15+(15+((15+((15+(15<<4))<<4))<<4))<<4))<<4))<<4))<<2));
    // 用好的 mask 来获得 4 位 4 位间隔的
    // mask = (0b11111111) ...

    // mask = 0xff+(0xff<<8)+(0xff<<16)+(0xff<<24)

    //return ((255+(255<<8)+(255<<16)+(255<<24))&x)<<4 | ((255+(255<<8)+(255<<16)+(255<<24))&(x>>4));
    // 绷不住了，竟然允许赋值，那还说啥了
    int mask = 15+(15<<8)+(15<<16)+(15<<24); // 间隔掩码

    return ((mask&x)<<4) | ((mask&(x>>4)));
}

```

思路：入股允许复制就很简单了。我们可以想象一下，这就是两个“波”的叠加。所以只用合适的“筛”，过滤一下，就好了。我们考虑间隔掩码，利用=赋值，得到 `mask = 15+(15<<8)+(15<<16)+(15<<24);` // 间隔掩码

一切就好做了。

P7 secondLowestZeroBit

```
// P7
/*
 * secondLowestZeroBit - return a mask that marks the position of the second least significant 0 bit
 *   Examples: secondLowestZeroBit(0xFFFFF0) = 0x4, secondLowestZeroBit(0x7FFFFFFF) = 0
 *             secondLowestZeroBit(-1) = 0
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 8
 *   Rating: 4
 */
int secondLowestZeroBit(int x) {

    // 取反就是
    // x & (-x) 最低的 1 位代表的值
    // 树状数组经典 trick, lowbit
    // return (~x) & ((~x)+1);
    // 上面求的是第 1 个 0, 题目要第 2 个 0
    // 取反后变成找第 2 个 1: 先 x&(x-1) 抹掉最低的 1, 再 lowbit 一次
    x=~x;
    x=x&(x+1);
    return x&(~x+1);
}
```

思路: inspired by 树状数组的 lowbit 算法 ($x \& (-x)$), 是先用 $v \& (v-1)$ 清除最低的 1, 再取 lowbit 。

P8 oddParity

```
// P8
/*
 * oddParity - return the odd parity bit of x, that is,
 *   when the number of 1s in the binary representation of x is even, then the return 1, otherwise
 *   return 0.
 *   Examples: oddParity(5) = 1, oddParity(7) = 0
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 56
 *   Rating: 5
 */
int oddParity(int x) {
    // 呃啊, 显然要折叠信息
    /// x ^ (x >> 16), 虽然最高的 16 位我删不掉, 但不重要, 我的后 16 位已经折叠了, 有足够的 32 位信息了

    // 如果能赋值那还好说???
    x=x^(x>>16);
    x=x^(x>> 8);
    x=x^(x>> 4);
    x=x^(x>> 2);
    x=x^(x>> 1);
}
```

```

    return !(x & 1);
}

```

思路：显然一个难点就是 op 限额。不难想到，oddparity，其实只有 01，如果合起来叠加来看，其实 odd/even 信息不会损失。我们可以按二分合并。显示前 16 和后 16 异或，奇偶不变（注意，我们如果只关心后 16 位，那么前 16 位没有消为 0 就问题不大），信息都在后 16 位，如此再 8 位，4，2，1。同理可以继续 $x = x \wedge (x \gg 1)$ ；到这一步，最后只用通过 01 判断即可，注意，前 31 位可能有若干无意义的，所以先得与上 1

```

return !(x & 1);

```

P9 rotateRightBits

```

// P9
/*
 * rotateRightBits - rotate x to right by n bits
 * you can assume n >= 0
 * Examples: rotateRightBits(0x12345678, 8) = 0x78123456
 * Legal ops: ! ~ & ^ | + << >>
 * Max ops: 16
 * Rating: 5
 */
int rotateRightBits(int x, int n) {
// return ((x>>n) ) & (~(((1<<n)+ (~1+1))<<(32+(~n+1))));
// 总会溢出
// anyway
// 不管了

// n = n & 31; // 取余数

// 上面只是逻辑右移，掉出去的低位没接回高位
// 而且 n=0 时 32-n 会移 32 位，x86 上等于没移，炸了 -> 拆成先移 31-n 再移 1

    int mask;
    n=n & 31; // 取余数
    mask=~(((1<<31)>>n)<<1); // 高 n 位是 0，其余是 1，消掉算术右移补的 1
    return ((x>>n)&mask) |(((x<<(31+(~n+1)))<<1);
//防止算术右移高位 11111 填补恶心到我们
}

```

思路：把 n 先取余数是一个好的技巧。至少会避免掉很多麻烦。最终肯定是一个或的形式。右移可能会在高位莫名其妙多出很多 1，所以经典来一个高位全 00 低位全 1 的 mask 就可以了。低位记得也要通过左移模拟这个 rotate 然后如果是 n=0 会炸，所以拆两步。

P10 roundEvenPow2

```

// P10
// ac
/*
 * roundEvenPow2 - round nonnegative x to the nearest multiple of 2^n.
 * If x is exactly halfway between two multiples, choose the multiple whose
 * quotient by 2^n is even.

```

```

* You can assume 0 <= x <= 0xffffffff and 1 <= n <= 16.
* Examples: roundEvenPow2(10, 2) = 8, roundEvenPow2(14, 2) = 16
* Legal ops: ! ~ & ^ | + << >>
* Max ops: 24
* Rating: 5
*/
int roundEvenPow2(int x, int n) {
    // i MADE IT!!!!
    return ((x+ (1 <<(n+ (~1+1))))+ ((x&(1<<n))>>n)+(~1+1)>>n)<<n;
}

```

思路： 这道题就是硬搞。何为 (2^n) 的倍数，就是后面都是 0；如何要四舍五入？四舍五入到整数的一个 idea 就是加上 0.5 再取整数部分，这里也是类似的想法。首先毋庸置疑，最后肯定是一个 $\ll n$ 的形式，那么之前就是一个 $\gg n$ ，这个 $\gg n$ 肯定就要有，同时在中间点还需要变得离偶数那个更近。

普通四舍五入只需要先加上 2^{n-1} ，但这里中点时不能无脑向上。所以稍微改成加 $2^{n-1}-1$ + function{当前商的奇偶性}，而 $((x \& (1 \ll n)) \gg n)$ 正好就是 $x \gg n$ 的最低位，也就是当前商是奇数还是偶数。这样商为偶数时，中点不会进位；商为奇数时，中点会进位到下一个偶数。

P11 midpointTowardFirst

```

// P11
/*
 * midpointTowardFirst - return the exact mathematical midpoint (x+y)/2
 * without overflow. If the exact midpoint lies halfway between two
 * integers, choose the adjacent integer that is closer to the first
 * argument x.
 * Examples: midpointTowardFirst(4, 7) = 5,
 *            midpointTowardFirst(7, 4) = 6
 * Legal ops: ! ~ & ^ | + << >>
 * Max ops: 32
 * Rating: 5
 */
int midpointTowardFirst(int x, int y) {
    // return ((x>>1)+(y>>1)) + (((x&1)^(y&1))) & ((y + (~x+1))>>31));

    // return (x&y) + ((x^y)>>1) + (((x^y)&1)&((y+~x+1)>>31));
    // 问题出在 y-x, 大正- 大负还会溢出 mmd 会溢出 (INT_MAX - INT_MIN)
    // 异号: x>y 当且仅当 y 是负的, 直接看 y 的符号
    // 同号: 减法不会溢出, 还是看 y-x 的符号
    int difference_sign = (x^y)>>31;
    //符号差异
    int sub = (difference_sign&y)|(~difference_sign&(y+~x+1));
    return (x&y) + ((x^y)>>1) + (((x^y)&1)&(sub>>31));
}

```

思路： // return (x+y>>1)) + (((x&1)^(y&1))) & ((y + (~x+1))>>31)); 就是这种还是比较好写的，就是处理到

更靠近谁的时候，我们只需要同时考虑 和的奇偶以及 xy 大小关系。 那么问题出在哪里呢？就是 y 与 x 操作的时候，大正数大负数的+/-总归会溢出。所以我们不如利用更加讨巧的方式 1. 重写求和除 2 为 $(x \& y) + ((x \oplus y) \gg 1)$ ； 那么靠近谁呢？就是通过符号判断来处理。 如果都是同号的，那就不如加减处理。 如果是异号的，则直接可以通过符号位判断出来谁打谁小。 具体的，我们先考虑 differencesign， 再去算 sub； 如果是负号的话， 那么 difference_sign 是 11111111。。。1111， 补码为 0， 那个有 y-x 的部分就可以被抛弃了。

P12 isBetweenEitherOrder

```
// P12
/*
 * isBetweenEitherOrder - return 1 when x lies in the inclusive interval whose
 * endpoints are a and b. The endpoints may be given in either order.
 * Example: isBetweenEitherOrder(5, 8, 3) = 1.
 * Legal ops: ! ~ & ^ | + << >>
 * Max ops: 48
 * Rating: 7
 */
int isBetweenEitherOrder(int x, int a, int b) {
// return (!(x^a)) | (!(x^b)) | ((~((x+(~a+1))>>31)+1) ^ (~(x+(~b+1)) >>31)+1));
// : x-a 会溢出
// 在区间外 = 同时小于 a,b 或者 同时大于 a,b

// int signa = (a>>31)&1;
// int signb = (b>>31)&1;
// //负数为1, 正数为0
// int signx=(x>>31)&1;

// int res_1 = (signa^signb) | (!(signa^signx));
// // a,b 同号, 并且 a, x 异号, 就一定是 0 | 0 =0
// // res_1 只有等于 0 才有意义

//回到最初的语句
//return (!(x^a)) | (!(x^b)) | ((~((x+(~a+1))>>31)+1) ^ (~(x+(~b+1)) >>31)+1));
//只要修正减法就是对的了!

//int delta_a = (x+(~a+1))

//int delta_a = (((x>>31)&1)^((a>>31)&1)) (~((x+(~a+1))>>31)+1);
//int delta_b = (((x>>31)&1)^((b>>31)&1)) (~((x+(~b+1))>>31)+1);
// 先把正-负或者负-正的情况给特判了

//剩下如出一辙

//return (!(x^a)) | (!(x^b)) | (delta_a ^ delta_b);
// diff_a: 异号为全1(-1), 同号为0
int diff_a = (x^a)>>31; int diff_b = (x^b)>>31;
```

```

// 异号直接取 x 的符号(若 x 负则全 1); 同号时做减法不会溢出, 取减法符号
int sign_a = ((diff_a&x) | (~diff_a&(x+(~a+1))))>>31;
int sign_b = ((diff_b&x) | (~diff_b&(x+(~b+1))))>>31;

// 端点相等, 或者两侧符号严格一正一负
return (!(x^a)) | (!(x^b)) | ((sign_a^sign_b)&1);
}

```

思路: 如果是`//return (!(x^a)) | (!(x^b)) | ((~((x+(~a+1))>>31)+1) ^ (~((x+(~b+1))>>31)+1));` 这个很容易写出, 特判端点情况+作差符号相反。但这里又出了问题, 减法的时候, 会有溢出。我们老样子。就是考虑作差后右移, 就是如果为负数, 最后得到的是`~0`, 再`~`一下回到 0, 会让“与”右边的减法失效。

P13 mul5Sat

```

// P13
/*
 * mul5Sat - return x*5, and if x*5 overflow, change the result to
 * INT_MAX(0x7fffffff) or INT_MIN(0x80000000) correspondingly
 * Examples: mul5Sat(1) = 0x5, mul5Sat(0x40000000) = 0x7fffffff
 * Legal ops: ! ~ & ^ | + << >>
 * Max ops: 30
 * Rating: 7
 */
int mul5Sat(int x) {
// return x+(x<<2);
// x*5 = x+(x<<2), 两种溢出:
// 1. 左移 2 位就溢出了: 移回来和 x 不一样
// 2. 相加溢出: 结果和 x 符号不同
// 溢出了就按 x 的符号选 INT_MAX / INT_MIN
    int x4 = x<<2;

// 第一次判*4 溢出与否
    int over = !((x4>>2)^x);

    int res= x4+x;

    over = over | (((res^x)>>31)&1);

    int mask = ~over+1; // 溢出变全 1, 不溢出全 0
    int INF = (1<<31)^(~(x>>31));

    return ((~mask)&res)|(mask&INF); // !!想了好久
}

```

思路: 先算 `x*4`, 再加 `x` 得到 `x*5`。分别检测左移（除以 4 是否相等）和加法（是否碰到了届界限, 超过了就会异号）是否溢出, 最后依据 `x` 的符号去构造 `inf` 对应的 `mask`, 这个就需要一点技巧, 可以看到巧妙利

用了取反。

P14 classifyAdd3

```
// P14
/*
 * classifyAdd3 - classify the exact mathematical sum x+y+z.
 * Return 1 if the sum is greater than INT_MAX, -1 if it is less than
 * INT_MIN, and 0 otherwise. You may not use a wider integer type.
 * Legal ops: ! ~ & ^ | + << >>
 * Max ops: 52
 * Rating: 7
 */
int classifyAdd3(int x, int y, int z) {
// return 14;
// 分两步加，每一步记一下溢出方向：正溢出 +1，负溢出 -1
// 两步加起来 >0 就是大于 INT_MAX, <0 就是小于 INT_MIN, =0 说明没溢出或者正好抵消
    int sum1 = x + y;
    int sum2 = sum1 + z;
//最烦的是这个溢出
    // 计算 x+y 的溢出

    // 再计算 (x+y)和 z 求和的溢出
    int c1 =
        (((~x) & (~y) & sum1) >> 31) & 1)
        + ((x & y & (~sum1)) >> 31);

    //对称的

    int c2 =
        (((~sum1) & (~z) & sum2) >> 31) & 1) +
        ((sum1 & z & (~sum2)) >> 31);

    int c = c1 + c2;

    return (c >> 31) | (!!c);
}
```

思路：这道题的难点在于：即使中间发生溢出，最终的数学结果也未必溢出，因为后续加法可能将其抵消。

思路是分两步加法，同时记录每一步的溢出方向。对于两个数相加，如果都是正数却得到负数，就是正溢出，记作 +1；如果都是负数却得到非负数，就是负溢出，记作 -1；否则记作 0。值得注意的就是说，因为 32 位整数的溢出本质上是模对(2^{32}) 取模。我们的溢出实际就是进位胡哦这借位，所以可以用一个额外的数值去记录这个。最终 c 的符号就也很简单，正数返回 1，负数返回 -1，零返回 0。

P15 floatScaleThreeHalves

```
// P15
/*
 * floatScaleThreeHalves - Return bit-level equivalent of expression f*3/2 for
 *   floating point argument f.
 *   Both the argument and result are passed as unsigned int's, but
 *   they are to be interpreted as the bit-level representation of
 *   single-precision floating point values.
 *   Use round-to-nearest-even. Preserve the sign of both +0 and -0.
 *   When argument is NaN, return argument.
 *   Legal ops: Any integer / unsigned operations incl. ||, &&. also if, while
 *   Max ops: 60
 *   Rating: 7
 */
unsigned floatScaleThreeHalves(unsigned uf) {
// return 15;
    unsigned sign = uf & 0x80000000;
//1 000 0000 0000 0000 0000 0000 0000

    unsigned exp = (uf >> 23) & 0xff;//eponet

//0111 1111 1111 1111 1111 1111 1111
    unsigned frac = uf & 0x7fffffff;//fraction

//特判
    if (exp == 0xff) return uf; // inf 和 nan 直接跑掉

// denormalized: 没有隐藏的 1, 尾数直接 *3/2
    if (exp == 0) {
        //unsigned res = frac+ (frac<<1);
//现在能乘法了, 没必要
        unsigned res = frac*3;
        int round = ((res & 3) == 3);
        // 3 类型还是 6 类型

        //round to even!!!
        res = (res >> 1) + round;

        if (res >= 0x800000) return sign | (1 << 23) | (res & 0x7fffffff);// 进位情况
        return sign | res;
    }

// normalized: 补上那个隐藏的 1, 再 *3
    unsigned res = (frac | 0x800000) * 3;
    if (res >= 0x2000000)
    {
        exp = exp + 1;// 到 2^25 了, 多右移一位, 阶码+1
    }
}
```

```

    int round = ((res & 3) > 2) || (((res & 3) == 2) && ((res >> 2) & 1));
    res = (res>>2)+round;
}
else
{
    int round = ((res & 3)==3);
    res = (res>>1)+round;
}
// 舍入之后又进位到 2^24
if (res >= 0x1000000) {
    res = res>>1;
    exp = exp+1;
}
if (exp >= 0xff) return sign | 0x7f800000; // 溢出变 inf
return sign | (exp << 23) | (res & 0x7fffff);
}

```

思路：先拆出 `exp`、`frac` 和 `sign`。特殊值先跑掉，NaN 和 `inf` 直接原样返回。然后就是 `normalized` 和 `denormalized` 分开处理，区别就在于有没有那个隐藏的 1。对于 `denormalized`，没有隐藏的 1，直接让 `frac` 乘 3 再除 2 就好了。但这里不能无脑右移，因为还需要 **round to even**！右移一位的时候，如果被舍弃的是 1，且留下来的最低位也是 1，就要再加 1。也就是看 `res` 的最后两位是不是 11。如果结果到了 `0x800000`，说明已经进入 `normalized` 的范围，直接把对应的阶码和 `frac` 拼回去。正负 0 也会在这个过程中保留下来。对于 `normalized`，先补上隐藏的 1，再乘 3。

这里要注意，我们最后只保留 24 位有效数字。如果 `res` 已经到了 `0x2000000`，说明需要右移两位，同时 `exp` 加 1；否则右移一位就够了。右移两位的时候，低两位大于 10 就进位，小于 10 不进位，正好是 10 则看保留下来的最低位，奇数才进位。这样中点就会靠向偶数。最后还有一个容易漏掉的地方：舍入本身也可能带来进位！如果 `res` 到了 `0x1000000`，还得再右移一位、`exp` 加 1。最后检查阶码有没有到 `0xff`，到了就变成对应符号的 `inf`，否则把 `sign`、`exp` 和 `frac` 或一下，拼回去即可。

P16 floatRoundEven

```

// P16
/*
 * floatRoundEven - round the floating-point value represented by uf to the
 * nearest integer, with halfway cases rounded to the even integer. Return
 * the bit-level representation of that integer as a single-precision float.
 * If rounding produces zero, preserve the input sign; thus a negative
 * value that rounds to zero returns -0. When uf is NaN or infinity,
 * return uf unchanged.
 * Legal ops: Any integer / unsigned operations incl. ||, &&. also if, while
 * Max ops: 65
 * Rating: 10
 */
unsigned floatRoundEven(unsigned uf) {
// return 16;
// 1000 0000 0000 0000 0000 0000 0000
    unsigned sign = uf&0x80000000;

```

```

// 1111 1111
unsigned exp=(uf>>23)&0xff;//高位乱七八糟的不要
int e =exp-127;//真实的指数, in 十进制来看, it is a number

if(exp == 0xff)return uf;// inf 和 nan 原样返回
if (e<-1)return sign;// |f| < 0.5 舍入到 0, 保留符号位
if (e== -1) {
    if (uf& 0x7fffff) return sign|0x3f800000;// 大于 0.5 进位到 1.0
    return sign;// 正好 0.5 向偶数舍入到 0
}
if(e>=23)return uf;// 无论如何这个指数都能把他弄成一个 integer, 直接返回

// 和下面一题致
int shift = 23-e;
int mask = (1 << shift) - 1;
// 这里也直接处理了, 到底需不需要进位到阶码
int half = 1 << (shift - 1);//用于判断四舍五入
int low = uf & mask;// 取出被舍弃的小数部分
int round = (low > half) || ((low == half) && ((uf >> shift) & 1));// 如出一辙
return (uf & ~mask)

    //尾部到底 frac 后面大概率都是 0000, 就是 mask 都是处理掉

    + (round << shift);
    // 四舍五入在 frac 的中间部分补上
}

```

思路: 由 `exp` 确定小数点位置。先拆出 `sign` 和 `exp`, 得到真实指数 `e=exp-127`。NaN 和 `inf` 这种 `cornercase` 先原样返回, 剩下的按小数点在哪里分类就好了。如果 `e<-1`, 那么绝对值小于 0.5, 直接回到 0, 但符号位得留下来。如果 `e== -1`, 那么绝对值就在 [0.5,1) 里面。这里正好 0.5 要回到 0, 因为 0 是偶数; 只要 `frac` 不是 0, 说明大于 0.5, 就回到对应符号的 1.0。如果 `e>=23`, 那小数点已经到所有有效位的后面, 本身就是整数, 直接返回。

最需要处理的就是 $0 \leq e < 23$ 。我们先算 `shift=23-e`, 这就是需要清掉的低位数。然后用 `mask` 取出这些低位, 得到 `low`, 再构造 `half` 作为中点, 就是四舍五入的阈值。`low>half` 当然要进位; `low==half` 的时候, 只有保留下来的最低位是 1 才进位, 这样才能靠近偶数。最后先清掉低位, 就是加上 `round<<shift`。这个加法很讨巧, `frac` 发生进位, 则会自然传到阶码, 不用再单独拆开处理。负数亦然。

P17 float_i2f

```

// P17
/*
 * float_i2f - Return bit-level equivalent of expression (float) x.
 *   Result is returned as unsigned int, but
 *   it is to be interpreted as the bit-level representation of a
 *   single-precision floating point values.
 *   Legal ops: Any integer / unsigned operations incl. ||, &&. also if, while
 *   Max ops: 40

```

```

*   Rating: 10
*/
unsigned float_i2f(int x) {
//   return 17;
// 0100 0000
    if (x == 0) return 0;
    if (x == 0x80000000) return 0xc0000000; // INT_MIN 绝对值溢出，单独特判

    unsigned sign = 0;
    if (x < 0)
    {
        sign = 0x80000000;
        x = -x;
    } // 统一处理正数

    // 找最高的 1
    int exp = 30;
    while(!((x >> exp)&1))--exp;

    // 不超过 23 位，精度仍在，，直接左移对齐
    if(exp <= 23) return sign|
        ((exp + 127) << 23)|
        ((x <<(23-exp)) & 0x7fffff);

    // 超过 23 位
    int shift = exp - 23;
    int mask = (1<<shift)-1;

    int half = 1<<(shift-1); // for 四舍五入

    int low = x & mask;

    int round = (low > half) || ((low == half) && ((x >> shift) & 1)); // 模拟四舍五入的两种情况
    x = (x>>shift) + round;

    // 针对进位的特判!! 舍入导致进位到 2^24，需要右移一位
    if(x>>24)
    {
        x=x>>1;
        exp=exp+1;
    }
    return sign|
        ((exp + 127) << 23)|
        (x & 0x7fffff);
}

```

```
}
```

思路：先处理零、边界值特殊情况。然后一个 **trick** 的点就是全都划归到正数。然后注意 $8+23$ ， 我们要注意，1. 四舍五入 2. 到底指数位是多少！精度能保留的 23 是简单的， 更难的是需要处理精度损失的四舍五入，以及四舍五入带来的进位！ $\text{exp} > 23$ ，这时候就必须舍弃低位。先算 $\text{shift} = \text{exp} - 23$ ，再用 **mask** 取出被舍弃的部分 **low**，以及 $\text{half} \ll (\text{shift} - 1)$ 。复用之前逻辑： $\text{low} > \text{half}$ 就进位； $\text{low} = \text{half}$ 则看 $x \gg \text{shift} \& 1$ ，也就是保留部分的最低位。它是奇数才加 1，偶数就留下来。这里要看完整的 **low**，光看被舍弃的最高一位还不够。最后就如同之前一样，拼起来！

P18 bitCount

```
// P18
/*
 * bitCount - return count of number of 1's in the binary representation of x
 * Examples: bitCount(5) = 2, bitCount(7) = 3
 * Legal ops: ! ~ & ^ | + << >>
 * Max ops: 40
 * Rating: 10
 */
int bitCount(int x) {

// int res = 0;
// return 18;

// 分治：相邻 1 位两两相加 -> 每 2 位存个数；再相邻 2 位相加 -> 每 4 位存个数 .....
int mask_1 = 85+(85<<8); // 0x5555
int mask_2 = 51+(51<<8); // 0x3333
int mask_4 = 15+(15<<8); // 0x0f0f
mask_1 = mask_1+(mask_1<<16);
mask_2 = mask_2+(mask_2<<16);
mask_4 = mask_4+(mask_4<<16);

x=(x&mask_1)+((x>>1)&mask_1);
x=(x&mask_2)+((x>>2)&mask_2);
x=(x+(x>>4))&mask_4; // 4 位最多是 8，加起来不会溢出到隔壁，可以先加再 mask
x=x+(x>>8);
x=x+(x>>16);
return x&63; // 最多 32 个，低 6 位够了
}
```

思路：Inspired by 线段树，我们可以使用分治累加。先统计相邻 2 位中有多少个 1，再将相邻的两个 2 位区域合并，得到每 4 位中 1 的个数。以此类推，逐步合并为 8、16、32 位，最终得到整个整数中 1 的总数。

一切都是要选取好的 **mask**。比如第一个就是要 $64+16+4+1$ 以及左移 8 位的它的和达到 16 位，最后达到 32 位。我们也就完成第一步的求和。以此类推，逐渐浓缩，最后到 1 位。每次通过 **&** 提取对应区域的计数，再通过移位和加法完成合并。注意，**&** 在这里并不是直接统计 1，而是筛选出需要保留的位，避免相邻区域互相干扰。

最后，32 位整数最多只有 32 个 1，因此只需要 6 位就能表示最终答案。返回 `x & 63`，提取低 6 位即可

P19 bitReverse

```
// P19
/*
 * bitReverse - Reverse bits in an 32-bit integer
 *   Examples: bitReverse(0x80000004) = 0x20000001
 *              bitReverse(0x7FFFFFFF) = 0xFFFFFFF
 *   Legal ops: ! ~ & ^ | + << >>
 *   Max ops: 34
 *   Rating: 10
 */
int bitReverse(int x)
{
    //归并排序?

    // 上下半区交换
    // 4444 交换?

    //递归!

    int mask_16 = (1<<16) + (~1+1);
    x=(x>>16) & mask_16|((x&mask_16)<< 16);

    int mask_8 = 255+ (255<<16) ;

    x = ( (x>>8) & mask_8 ) | ((x&mask_8)<<8);

    int mask_4 = 15 + (15<<8) + (15<<16) + (15<<24);
    x = ((x>>4) & mask_4 ) | ((x&mask_4)<<4);

    int mask_2 = 3 + (3<<4) + (3<<8) + (3<<12) + (3<<16) + (3<<20) + (3<<24) + (3<<28);

    x = ((x>>2) & mask_2 ) | ((x&mask_2)<<2);

    int mask_1 = 1 + (1<<2) + (1<<4) + (1<<6) + (1<<8) + (1<<10) + (1<<12) + (1<<14) + (1<<16) + (1<<18) +
    (1<<20) + (1<<22) + (1<<24) + (1<<26) + (1<<28) + (1<<30);

    x = ((x>>1) & mask_1) | ((x&mask_1)<<1);
```

```
return x;  
}
```

思路：这道题有点难。平衡树的某一个特殊应用“文艺平衡树”可以给我们启发。对于这样的区间反转，它能够 $\log n$ 的更新。是怎么做到的？它是一个二叉树代表区间，把翻转 **tag** 留在树上，每一步向下传递。那么这里，我们这里采用 **swap**，可以复刻这种思想。我们前 16 和后 16 可以两两交换，再者就是相邻的 8-8 两两交换，再就是 4-4 4-4 4-4 4-4 两两交换....(就如同平衡树的某个子树的左子树和右子树交换)。正如前面有一道题已经实现过的 44swap，我们这里需要的是不同尺寸的 **mask**。`int mask_16 = (1<<16) + (~1+1);`
`x=(x>>16) & mask_16|((x&mask_16)<< 16);`
这便是前 16 后 16 交换；再进一步，能够所有的前 8 和后 8 交换。